

Python for Quants

Interview Cheat Sheet

Data Structure Complexity

<code>list</code>	Access $O(1)$	Insert $O(N)$
<code>dict</code>	All $O(1)$	Hash-based
<code>set</code>	All $O(1)$	Membership
<code>deque</code>	Ends $O(1)$	Rolling windows
<code>heapq</code>	Min $O(1)$	Push $O(\log N)$

GIL Rules

- CPU-bound $\rightarrow$ multiprocessing
- I/O-bound (few) $\rightarrow$ threading
- I/O-bound (many) $\rightarrow$ `asyncio`
- GIL released during I/O, NumPy, Numba

Memory Optimization

- `__slots__` = 50% memory savings
- `category dtype` = 10x for strings
- Downcast: `int64` $\rightarrow$ `int32/int16`
- Generator = $O(1)$ memory

NumPy Speed Rules

- Vectorize: 100x faster than loops
- Pre-allocate arrays
- Slice = view, fancy index = copy
- `np.where` for conditionals
- `np.einsum` for complex ops

Pandas Traps

- `.apply()` is a LOOP (slow!)

- Use `.query()` for large DataFrames
- Pre-sort before multiple `groupby`
- Use `.loc[]` for assignment
- Avoid chained indexing

Classic Gotchas

- Mutable default: `def f(x=[])`
- Late binding: `lambda: i`
- Float precision: `0.1 + 0.2 != 0.3`
- Shallow copy: nested lists shared
- Generator exhaustion: iterate once

Decorator Patterns

<code>@timer</code>	Profile execution
<code>@lru_cache</code>	Memoization
<code>@retry</code>	Handle failures
<code>@property</code>	Computed attribute
<code>@contextmanager</code>	Resource cleanup

Type Hints

```
from typing import List, Dict, Optional

def func(
    trades: List[Dict[str, float]],
    n: Optional[int] = None
) -> float:
```

3-Second Answers

List vs Tuple?

“List mutable, tuple immutable. Tuple for dict keys.”

Dict complexity?

“ $O(1)$ average, $O(N)$ worst. Hash table.”

What is GIL?

“Lock preventing parallel threads. Use multiprocessing for CPU.”

View vs Copy?

“Slicing = view. Fancy indexing = copy.”

Why is apply slow?

“It’s a hidden Python loop, not vectorized.”

What is yield?

“Pauses function, returns generator. Lazy evaluation.”

Closure?

“Function remembering outer scope variables.”

__slots__?

“Prevents `__dict__`, saves 50% memory.”

Shallow vs deep copy?

“Shallow copies container, deep copies nested.”

@staticmethod vs @classmethod?

“Static: no self/cls. Class: receives cls.”

System Design Keywords

- Event-driven $\rightarrow$ decoupled components
- Queue $\rightarrow$ producer-consumer
- AsyncIO $\rightarrow$ concurrent I/O
- ProcessPool $\rightarrow$ parallel CPU
- LRU cache $\rightarrow$ expensive computations

Testing

- `pytest.approx()` for floats
- `np.allclose()` for arrays
- Property-based: `hypothesis`
- Never `== 0.3` for floats!